

■ **Track 1 Objective & Core Challenge:**

Standard OCR (Tesseract) drops below 20% accuracy on cursive Indian doctor handwriting. Track 1 implements a **Hybrid Vision AI Pipeline** combining OpenCV CLAHE & de-skew preprocessing, a fine-tuned **Microsoft TrOCR (Vision Transformer)**, and a **RapidFuzz Matcher** against PillSync's 253,973 Indian Medicine Catalog to extract structured prescriptions with >95% accuracy.

■ **Team Work Breakdown: Chanchal vs Rohan**

■■■ CHANCHAL (Engineer 1A) — Preprocessing & API	■■■ ROHAN (Engineer 1B) — Deep Learning & MLOps
• OpenCV CLAHE & Shadow Suppression: Adaptive contrast boosting and background division. • Auto De-skewing & Geometry: MinAreaRect contour detection to rotate tilted documents to 0°. • Document Segmenter: Line & word bounding box crop extraction. • 253k Catalog Matcher: RapidFuzz Levenshtein string matching for zero-misspelling safety. • FastAPI Integration: Merge into backend/app/services/ocr_service.py.	• Dataset Ingestion: RxHandBD (5,500+ handwriting samples) + IAM dataset formatting. • Synthetic Data Generator: Cursive Indian prescription augmentation loop. • TrOCR Fine-Tuning: PyTorch Seq2Seq training on microsoft/trocr-base-handwritten. • Evaluation Loop: Character Error Rate (CER ≤ 12%) & Word Error Rate (WER ≤ 18%). • INT8 ONNX Quantization: Sub-350ms CPU latency optimization.

■■ **Step-by-Step Training & Setup Instructions**

Step 1: Isolated Environment Setup

Navigate to the project root and set up the Track 1 workspace:

```
cd ai_training/track_1_vision
python -m venv venv
# Windows: .\venv\Scripts\activate | Linux/Mac: source venv/bin/activate
pip install torch torchvision transformers datasets accelerate evaluate jiwer opencv-python-headless rapidfuzz onnxruntime
```

Step 2: Training Execution & Validation

- Rohan runs python src/train_trocr.py (Fine-tunes model on GPU/CPU for 5 epochs).
- Rohan runs python src/evaluate.py to confirm CER ≤ 12% and WER ≤ 18%.
- Rohan exports lightweight ONNX model: python src/export_onnx.py.
- Chanchal connects cv2_preprocessor.py and fuzzy_catalog_matcher.py to backend/app/services/ocr_service.py.
- Verify end-to-end extraction with pytest tests/test_inference.py.

■ **The Master Prompt (Copy-Paste for AI Agent / IDE Execution)**

Give this complete prompt to any AI coding assistant or team member to execute Track 1:

[MASTER PROMPT FOR TRACK 1 EXECUTION]

Role: Lead Vision AI Engineer on PillSync Healthcare Platform.
Task: Implement Track 1 (Doctor Handwriting Vision AI & OCR Pipeline).
Requirements:

1. Build cv2_preprocessor.py (CLAHE + shadow removal + minAreaRect de-skewing).
2. Build document_segmenter.py (Line and word bounding box segmentation).
3. Fine-tune Microsoft TrOCR (microsoft/trocr-base-handwritten) on RxHandBD dataset.
4. Build fuzzy_catalog_matcher.py matching raw text against 253,973 Indian Medicines.
5. Quantize model to dynamic INT8 ONNX (<350ms CPU execution).
6. Merge inference engine into backend/app/services/ocr_service.py.

Quality Gates: CER ≤ 12%, WER ≤ 18%, Catalog Match Accuracy ≥ 95%, CPU Latency ≤ 350ms.
Proceed autonomously and write production-ready code.

■ Quality Evaluation Gates & Acceptance Criteria

Metric	Target Threshold	Verification Method
Character Error Rate (CER)	≤ 12.0%	500 test handwriting words via evaluate.py
Word Error Rate (WER)	≤ 18.0%	Full prescription line crops
Catalog Match Precision	≥ 95.0%	RapidFuzz matching against 253k Indian Catalog
CPU Inference Latency	< 350 ms	INT8 ONNX average over 50 test runs